**INDIAN INSTITUTE OF TECHNOLOGY BHILAI**

# LABORATORY REPORT

*Segmentation Lab (Modules 1 to 4)*

CSL301: Operating Systems


| | |
|---|---|
| **Name** | Rohit Raghuwanshi |
| **ID** | 12341820 |
| **Course** | Operating Systems (CSL301) |
| **Date** | 29 September 2026 |

## Module 1: Downward-Growing Stack Translation

The bound check runs first: if the offset is not below the limit, it prints SEGMENTATION FAULT. Otherwise, for a downward segment, Physical Address = Base - Offset (4700 - 100 = 4600).

### Code: mod1.c

```c
#include <stdio.h>

typedef struct {
    int id;
    const char *type;
    int base;
    int limit;
    int downward;   // 1 = grows downward (stack), 0 = upward
} Segment;

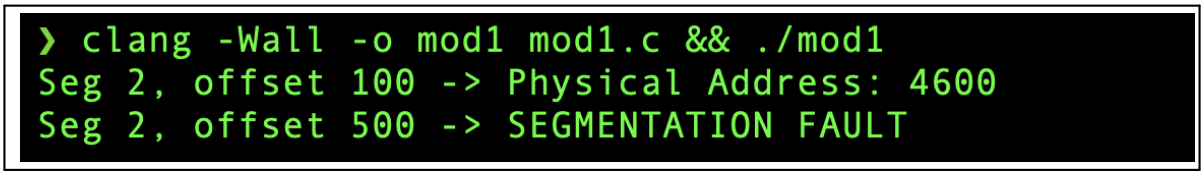
int translate(Segment s, int offset) {
    if (offset < 0 || offset >= s.limit) {
        printf("Seg %d, offset %d -> SEGMENTATION FAULT\n", s.id, offset);
        return -1;
    }
    int pa = s.downward ? s.base - offset : s.base + offset;
    printf("Seg %d, offset %d -> Physical Address: %d\n", s.id, offset, pa);
    return pa;
}

int main(void) {
    Segment stack = {2, "Stack (Downward)", 4700, 400, 1};
    translate(stack, 100);   // expect 4600
    translate(stack, 500);   // expect SEGMENTATION FAULT
    return 0;
}
```

### Command

```
$ clang -Wall -o mod1 mod1.c && ./mod1
```

### Output




*Output of mod1*

## Module 2: Multi-Ring Privilege Checker

The ring check runs before the R/W/X check: if cpu_ring > min_ring, the access traps with a Privilege Violation. Otherwise the request passes only if (req & perms) == req.

### Code: mod2.c

```
#include <stdio.h>

#define PERM_R 1
#define PERM_W 2
#define PERM_X 4

typedef struct {
    int id;
    const char *name;
    int min_ring;   // 0 = Kernel ... 3 = User
    int perms;      // bitmask of R/W/X
} Segment;

void check_access(int cpu_ring, Segment s, int req) {
    printf("Ring %d -> Seg %d (%s), min_ring %d, req %d, allowed %d: ",
           cpu_ring, s.id, s.name, s.min_ring, req, s.perms);

    if (cpu_ring > s.min_ring) {
        printf("TRAP: Privilege Violation\n");
        return;
    }
    if ((req & s.perms) != req) {
        printf("PERMISSION DENIED\n");
        return;
    }
    printf("SUCCESS\n");
}

int main(void) {
    Segment kcode = {0, "Kernel Code", 0, PERM_R | PERM_X};
    check_access(3, kcode, PERM_R);   // expect TRAP
    check_access(0, kcode, PERM_R);   // expect SUCCESS
    return 0;
}
```

### Command

```
$ clang -Wall -o mod2 mod2.c && ./mod2
```

### Output

```
❯ clang -Wall -o mod2 mod2.c && ./mod2
Ring 3 -> Seg 0 (Kernel Code), min_ring 0, req 1, allowed 5: TRAP: Privilege Violation
Ring 0 -> Seg 0 (Kernel Code), min_ring 0, req 1, allowed 5: SUCCESS
```

*Output of mod2*

## Module 3: Best-Fit and Worst-Fit Allocation

Both programs start with holes A (base 200, size 300) and B (base 700, size 200) and request 150. **Best fit** picks the smallest hole that fits (B), leaving a hole at base 850 of size 50. **Worst fit** picks the largest hole (A), leaving a hole at base 350 of size 150. The leftover hole starts at base + request and has size hole - request.

### Code: mod3_best.c (Best fit)

```
#include <stdio.h>

#define MAX_HOLES 10

typedef struct {
    int base;
    int size;
} Hole;

Hole holes[MAX_HOLES] = {
    {200, 300},   // Hole A
    {700, 200}    // Hole B
};
int hole_count = 2;

void print_holes(void) {
    for (int i = 0; i < hole_count; i++)
        printf("  Hole: Base %d (Size %d)\n", holes[i].base, holes[i].size);
}

int best_fit_allocate(int req_size) {
    int best = -1;

    // find the smallest hole that still fits the request
    for (int i = 0; i < hole_count; i++) {
        if (holes[i].size >= req_size &&
            (best == -1 || holes[i].size < holes[best].size))
            best = i;
    }

    if (best == -1) {
        printf("Allocation of %d failed: no hole large enough\n", req_size);
        return -1;
    }

    int alloc_base = holes[best].base;
    int leftover = holes[best].size - req_size;

    printf("Requested size: %d\n", req_size);
    printf("Chosen hole: Base %d (Size %d)\n", alloc_base, holes[best].size);

    if (leftover > 0) {
        holes[best].base += req_size;   // leftover becomes the new hole
        holes[best].size = leftover;
        printf("New leftover hole: Base %d (Size %d)\n",
               holes[best].base, holes[best].size);
    } else {
        for (int i = best; i < hole_count - 1; i++)
            holes[i] = holes[i + 1];
        hole_count--;
        printf("No leftover; hole removed\n");
    }
    return alloc_base;
}

int main(void) {
    printf("=== BEST FIT ===\n");
    printf("Holes before:\n");
    print_holes();
    best_fit_allocate(150);
    printf("Holes after:\n");
    print_holes();
    return 0;
}
```

### Command

```
$ clang -Wall -o mod3_best mod3_best.c && ./mod3_best
```

### Output

```
❯ clang -Wall -o mod3_best mod3_best.c && ./mod3_best
=== BEST FIT ===
Holes before:
  Hole: Base 200 (Size 300)
  Hole: Base 700 (Size 200)
Requested size: 150
Chosen hole: Base 700 (Size 200)
New leftover hole: Base 850 (Size 50)
Holes after:
  Hole: Base 200 (Size 300)
  Hole: Base 850 (Size 50)
```

*Output of mod3_best (Best fit)*

## Code: mod3_worst.c (Worst fit)

```
#include <stdio.h>

#define MAX_HOLES 10

typedef struct {
    int base;
    int size;
} Hole;

Hole holes[MAX_HOLES] = {
    {200, 300},   // Hole A
    {700, 200}    // Hole B
};
int hole_count = 2;

void print_holes(void) {
    for (int i = 0; i < hole_count; i++)
        printf("  Hole: Base %d (Size %d)\n", holes[i].base, holes[i].size);
}

int worst_fit_allocate(int req_size) {
    int worst = -1;

    // find the largest hole that fits the request
    for (int i = 0; i < hole_count; i++) {
        if (holes[i].size >= req_size &&
            (worst == -1 || holes[i].size > holes[worst].size))
            worst = i;
    }

    if (worst == -1) {
        printf("Allocation of %d failed: no hole large enough\n", req_size);
        return -1;
    }

    int alloc_base = holes[worst].base;
    int leftover = holes[worst].size - req_size;

    printf("Requested size: %d\n", req_size);
    printf("Chosen hole: Base %d (Size %d)\n", alloc_base, holes[worst].size);

    if (leftover > 0) {
        holes[worst].base += req_size;   // leftover becomes the new hole
        holes[worst].size = leftover;
        printf("New leftover hole: Base %d (Size %d)\n",
               holes[worst].base, holes[worst].size);
    } else {
        for (int i = worst; i < hole_count - 1; i++)
            holes[i] = holes[i + 1];
        hole_count--;
        printf("No leftover; hole removed\n");
    }
    return alloc_base;
}

int main(void) {
    printf("=== WORST FIT ===\n");
    printf("Holes before:\n");
    print_holes();
    worst_fit_allocate(150);
    printf("Holes after:\n");
    print_holes();
    return 0;
}
```

### Command

```
$ clang -Wall -o mod3_worst mod3_worst.c && ./mod3_worst
```

### Output

```
❯ clang -Wall -o mod3_worst mod3_worst.c && ./mod3_worst
=== WORST FIT ===
Holes before:
  Hole: Base 200 (Size 300)
  Hole: Base 700 (Size 200)
Requested size: 150
Chosen hole: Base 200 (Size 300)
New leftover hole: Base 350 (Size 150)
Holes after:
  Hole: Base 350 (Size 150)
  Hole: Base 700 (Size 200)
```

*Output of mod3_worst (Worst fit)*

## Module 4: Memory Compaction

compact_memory() moves each payload to the lowest free address with memmove(), updates its base, and clears the old bytes with memset(). "HELLO" (6 bytes with the terminator) moves to base 0, so "WORLD" lands at base 6.

### Code: mod4.c

```
#include <stdio.h>
#include <string.h>

#define RAM_SIZE 1000
#define MAX_SEGS 10

char RAM[RAM_SIZE];

typedef struct {
    int id;
    int base;
    int size;   // payload length including '\0'
} Segment;

Segment segs[MAX_SEGS];
int seg_count = 0;

void add_segment(int id, int base, const char *payload) {
    int size = strlen(payload) + 1;
    memcpy(&RAM[base], payload, size);
    segs[seg_count++] = (Segment){id, base, size};
}

void compact_memory(void) {
    // sort segments by base address (simple bubble sort)
    for (int i = 0; i < seg_count - 1; i++)
        for (int j = 0; j < seg_count - 1 - i; j++)
            if (segs[j].base > segs[j + 1].base) {
                Segment t = segs[j];
                segs[j] = segs[j + 1];
                segs[j + 1] = t;
            }

    int next_free = 0;
    for (int i = 0; i < seg_count; i++) {
        int old_base = segs[i].base;
        int size = segs[i].size;

        if (old_base != next_free) {
            memmove(&RAM[next_free], &RAM[old_base], size);

            // clear only old bytes not covered by the new location
            int clear_start = (old_base > next_free + size) ? old_base : next_free + size;
            int clear_end = old_base + size;
            if (clear_end > clear_start)
                memset(&RAM[clear_start], 0, clear_end - clear_start);

            segs[i].base = next_free;
        }
        next_free += size;
    }
}

void print_state(const char *title) {
    printf("%s\n", title);
    for (int i = 0; i < seg_count; i++)
        printf("  Seg %d: Base %d, Payload \"%s\"\n",
               segs[i].id, segs[i].base, &RAM[segs[i].base]);
}

int main(void) {
    memset(RAM, 0, RAM_SIZE);
    add_segment(0, 200, "HELLO");
    add_segment(1, 600, "WORLD");

    print_state("Before compaction:");
    compact_memory();
    print_state("After compaction:");

    printf("RAM[0..4]  = \"%.5s\"\n", &RAM[0]);
    printf("RAM[6..10] = \"%.5s\"\n", &RAM[6]);
```

```
    printf("Old positions cleared: RAM[200]=%d, RAM[600]=%d\n", RAM[200], RAM[600]);
    return 0;
}
```

### Command

```
$ clang -Wall -o mod4 mod4.c && ./mod4
```

### Output

```
❯ clang -Wall -o mod4 mod4.c && ./mod4
Before compaction:
  Seg 0: Base 200, Payload "HELLO"
  Seg 1: Base 600, Payload "WORLD"
After compaction:
  Seg 0: Base 0, Payload "HELLO"
  Seg 1: Base 6, Payload "WORLD"
RAM[0..4]  = "HELLO"
RAM[6..10] = "WORLD"
Old positions cleared: RAM[200]=0, RAM[600]=0
```

*Output of mod4*